

```
python
```

```
(self, creationid, creationseq):  
    'm a new Gene object!')  
    y constructor got a param: " + str(creationid))  
    ssigning that param to my id instance variable.  
    = creationid  
    imilarly, assigning to my sequence instance var:  
    uence = creationseq
```

```
(self):  
    y id is: " + str(self.id))
```

```
n(self):  
    y sequence len is: " + str(len(self.sequence)))
```

```
reating geneA:")  
342", "CATTGAC")
```

```
reating geneB:")  
4B", "TTACTAGA")
```

```
sking geneA to print_id():")
```

```
sking geneB to print_id():")
```

```
sking geneA to print_len():")  
)
```



Object Oriented Programming in Python

Most programs and techniques used so far follow a procedural approach. However, for large and complex projects with large teams, the object-oriented approach is invaluable. Python, being a versatile tool, allows for both object-oriented and procedural programming.



by Prof. M. Iqbal Bhat

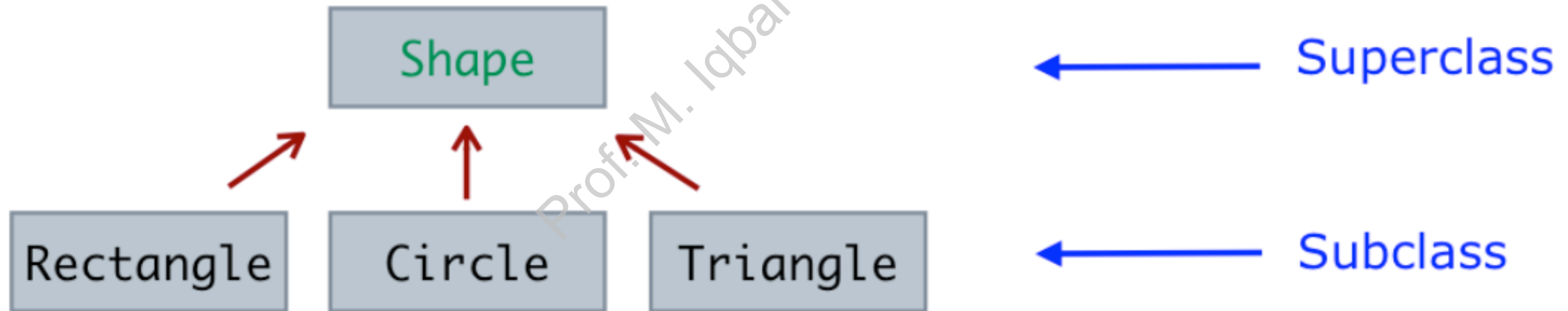
Inheritance

Inheritance allows one class (aka "the child class") to *inherit* the properties and methods of another class (aka "the parent class").

This powerful language feature helps us avoid writing a lot of the same code twice. It allows us to DRY (don't repeat yourself) up our code.

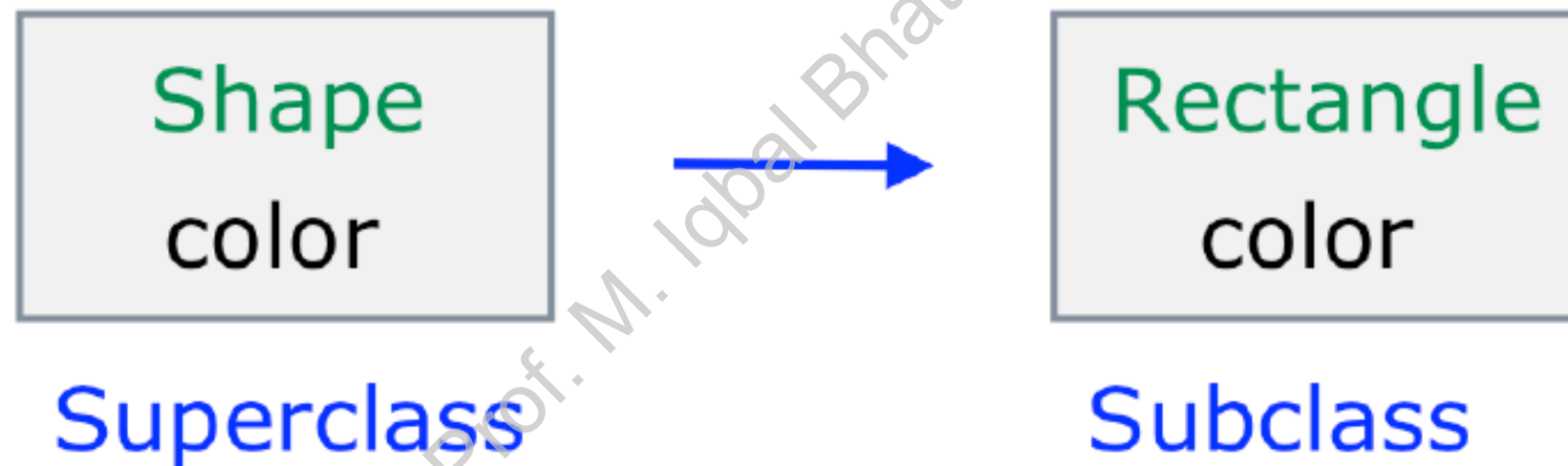
Inheritance

Class inheritance is designed to model relationships of the type "x is a y" (e.g. "a triangle is a shape")



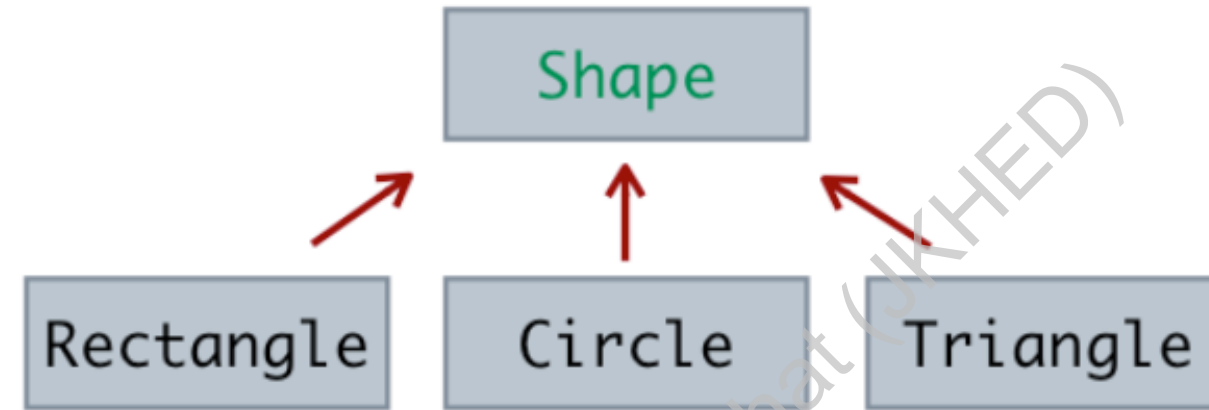
Inheritance

The functions and attributes of a superclass are inherited by a subclass.



An inherited class can override, modify or augment the functions and attributes of its parent class.

Creating Subclasses



Superclass

```
>>> class Shape:
...     pass
```

Name of superclass

Subclass

```
>>> class Rectangle(Shape):
...     def area(self):
...         return self.width*self.height
```

Creating Subclasses

Simple superclass

```
>>> class Shape:
...     def __init__( self ):
...         self.color = (0,0,0)
```

Simple subclass
inheriting from
Shape

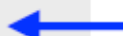
```
>>> class Rectangle(Shape):
...     def __init__( self, w, h ):
...         Shape.__init__( self )
...         self.width = w
...         self.height = h
...
...     def area( self ):
...         return self.width*self.height
```

Need to call
constructor
function in
superclass



```
>>> r1 = Rectangle( 10, 5 )
>>> print r1.width
10
>>> print r1.height
5
>>> print r1.area()
50
>>> print r1.color
(0, 0, 0)
```

Construct
object instance



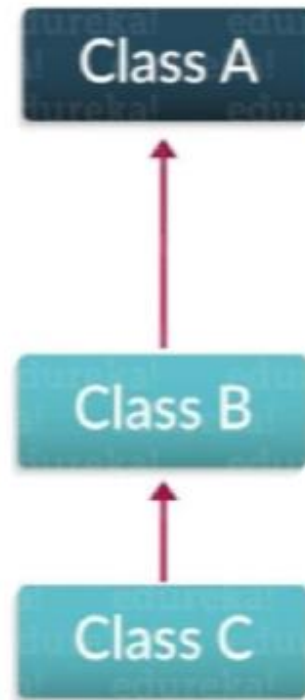
Inherited
attribute



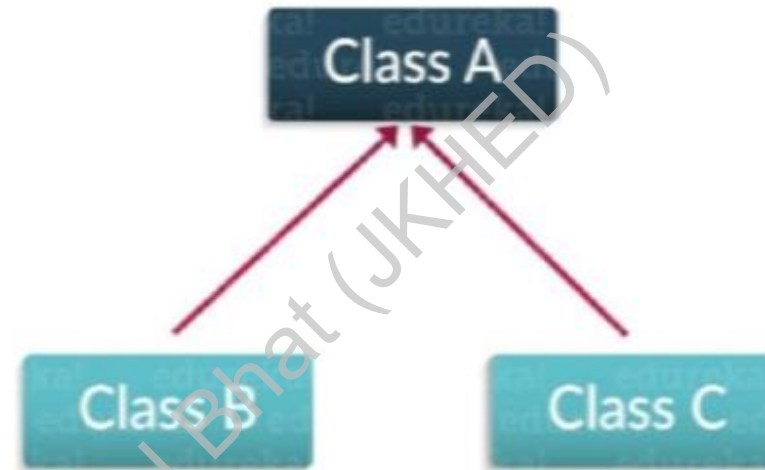
Types of Inheritance



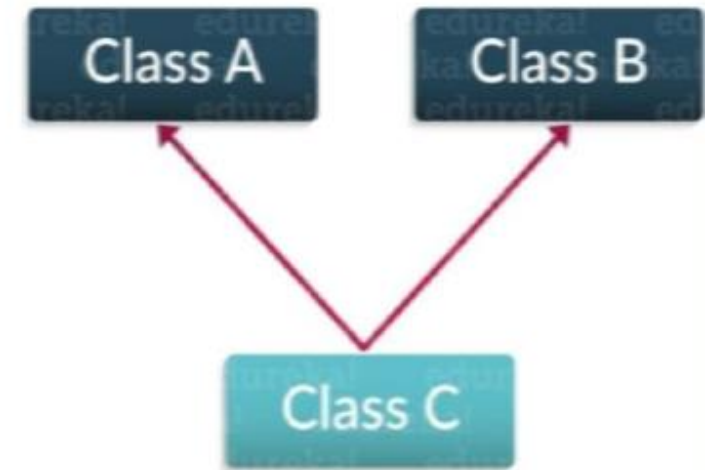
Single Inheritance



Multilevel Inheritance



Hierarchical Inheritance



Multiple Inheritance

Single Inheritance

Single level inheritance enables a derived class to inherit characteristics from a single parent class.

```
class employee1()://This is a parent class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
```

```
class childemployee(employee1)://This is a child class
    def __init__(self, name, age, salary,id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id
emp1 = employee1('harshit',22,1000)
print(emp1.age)
```



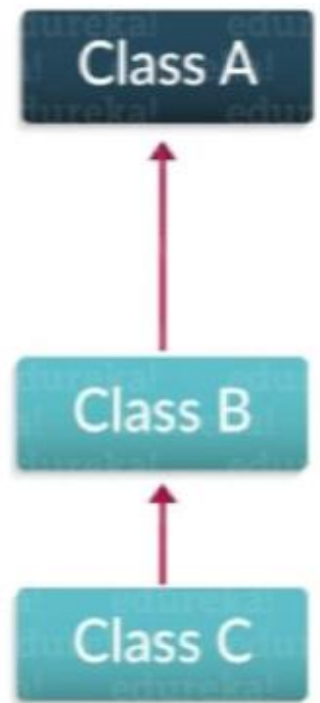
Single Inheritance

Multi-level Inheritance

Multi-level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class.

```
class employee(): // Super class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
class childemployee1(employee): // First child class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
class childemployee2(childemployee1): // Second child class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
emp1 = employee('harshit', 22, 1000)
emp2 = childemployee1('arjun', 23, 2000)

print(emp1.age)
print(emp2.age)
```



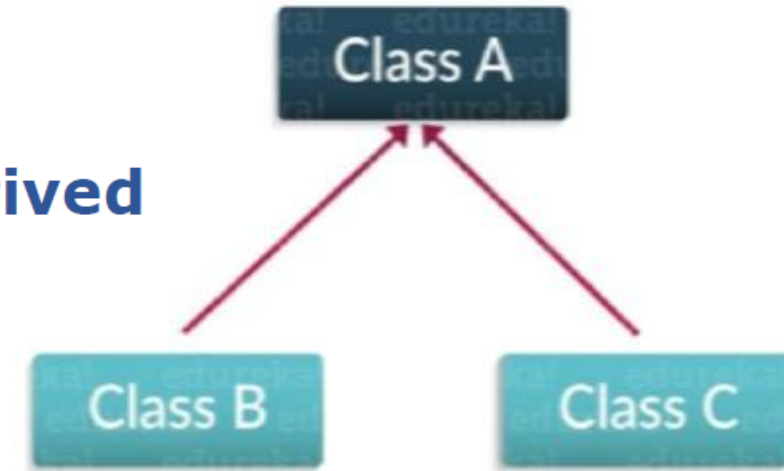
Multilevel Inheritance

Hierarchical Inheritance

Hierarchical level inheritance enables more than one derived class to inherit properties from a parent class.

```
class employee():  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary  
  
class childemployee1(employee):  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary  
  
class childemployee2(employee):  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary  
  
emp1 = employee('harshit', 22, 1000)  
emp2 = employee('arjun', 23, 2000)
```

//Hierarchical Inheritance



Hierarchical Inheritance

Multiple Inheritance

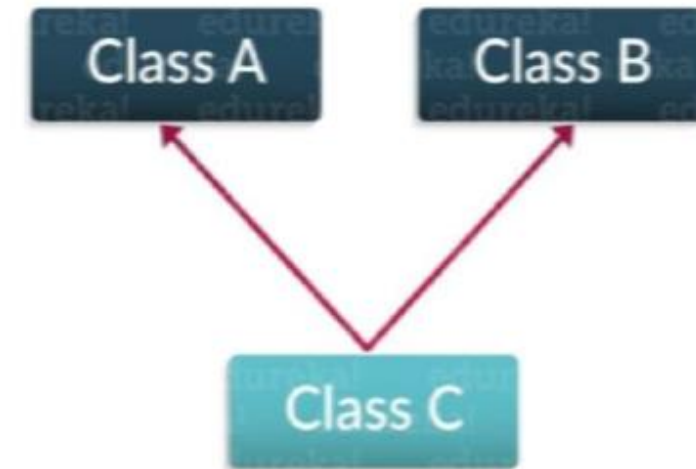
Multiple level inheritance enables one derived class to inherit properties from more than one base class.

```
class employee1(): //Parent class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

class employee2(): //Parent class
    def __init__(self, name, age, salary, id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id

class childemployee(employee1, employee2):
    def __init__(self, name, age, salary, id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id
```

```
emp1 = employee1('harshit', 22, 1000)
emp2 = employee2('arjun', 23, 2000, 1234)
```



Multiple Inheritance

Polymorphism

Prof. M. Iqbal Bhat (JKHED)

Polymorphism

Take a look at the Greek roots of "polymorphism".

- "poly" means "many"
- "morph" means "to change" or "form"
- Polymorphism is a fundamental concept in object-oriented programming (OOP) that allows objects of different classes to respond to the same method call in different ways.
- This means that a single method name can have different implementations depending on the type of object it is called on.
- Polymorphism makes code more flexible and reusable by allowing developers to write code that can work with different objects without knowing their exact type.

Polymorphism

```
# length of string
```

```
x = len('Geeks')
```

```
print(x)
```

```
# length of list
```

```
y = len([1, 2, 3, 4])
```

```
print(y)
```

Polymorphism Types:

Polymorphism is of two types:

- ❑ **Compile-time Polymorphism**
- ❑ **Run-time Polymorphism**

Prof. M. Iqbal Bhatt (JKUED)

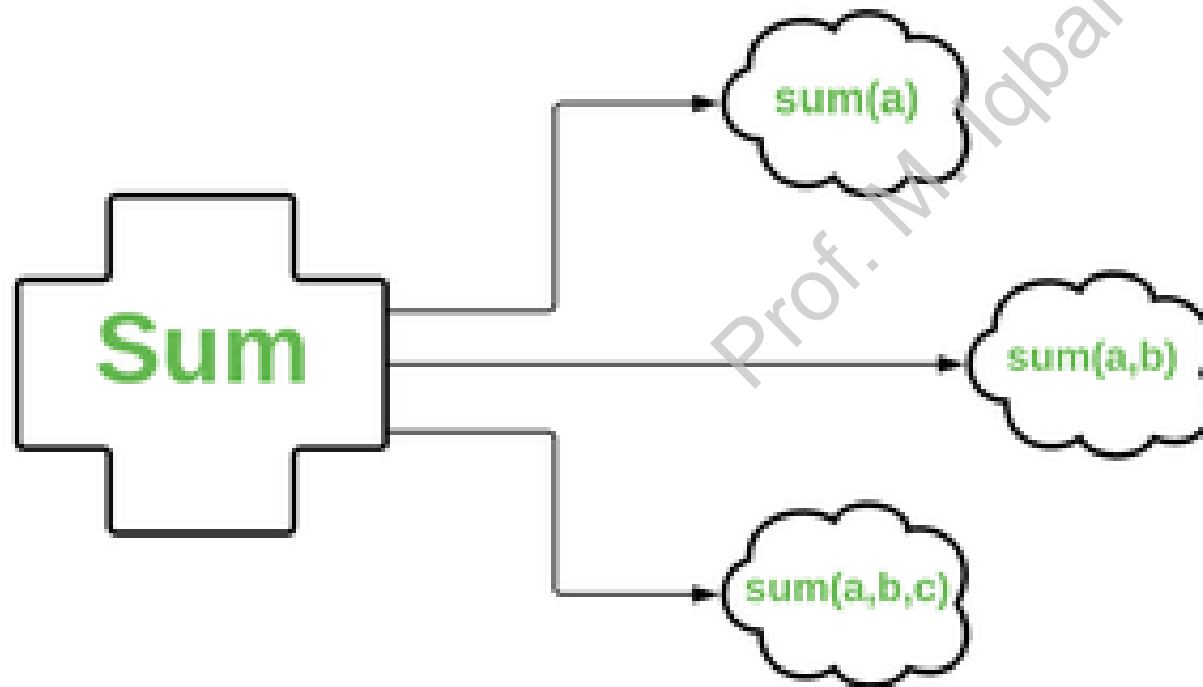
Compile-time Polymorphism:

A compile-time polymorphism also called as static polymorphism which gets resolved during the compilation time of the program. One common example is "method overloading"

- **Method overloading** refers to the ability of a class to have multiple methods with the same name but different parameters or different combinations of parameters. This allows for a more flexible and versatile code structure, enabling the same method name to handle different types of data or input combinations.
- Python, unlike many other programming languages, **does not** natively support method overloading. However, there are alternative approaches to achieve similar functionality, commonly referred to as "simulated method overloading"

Method Overloading:

- Given a single function `sum ()`, the number of parameters can be specified by you. This process of calling the same method in different ways is called method overloading



```
class Method_overloading:
    def sum(self, a=None, b=None, c=None):
        s = 0
        if a != None and b != None and c != None:
            s = a + b + c
        elif a != None and b != None:
            s = a + b
        else:
            s = a
        return s

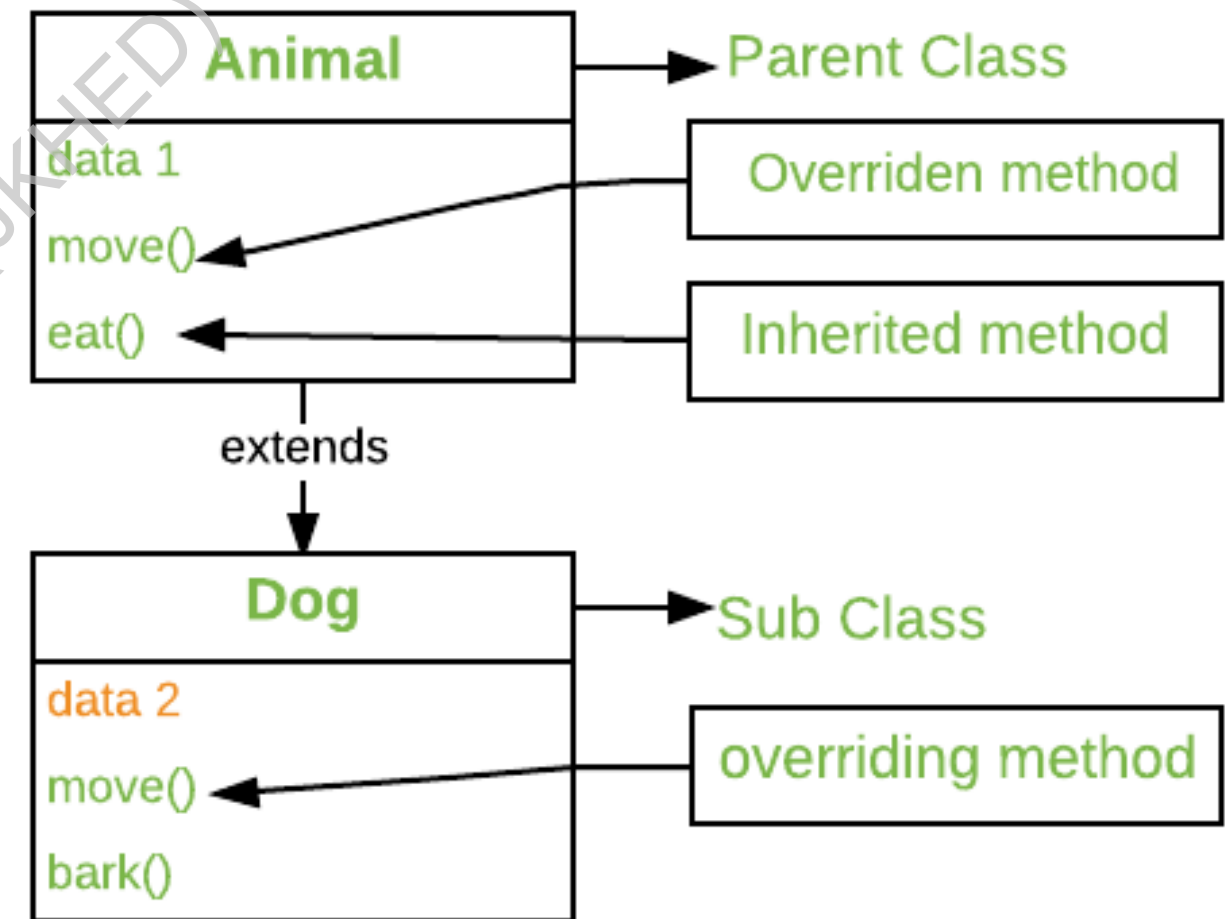
s = Method_overloading()
# sum of 1 integer
print(s.sum(1))
# sum of 2 integers
print(s.sum(a: 3, b: 5))
# sum of 3 integers
print(s.sum(a: 1, b: 2, c: 3))
```

Run-time Polymorphism:

- A Run-time Polymorphism is also called Dynamic Polymorphism where its gets resolved into the run time.
- Runtime polymorphism in Python is achieved through a mechanism called "method overriding" and is commonly associated with object-oriented programming (OOP).
- It allows objects of different classes to be treated as objects of a common base class. This concept is fundamental to achieving abstraction and encapsulation in Python.

Run-time Polymorphism: Method Overriding

- **Method overriding** is an ability of any object-oriented programming language that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes.
- When a method in a subclass has the same name, same parameters or signature and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class.



Run-time Polymorphism: Method Overriding

```
class Animal:
    def speak(self):
        pass # Abstract method

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"
```

```
def animal_sound(animal):
    return animal.speak()

# Creating objects of different classes
dog = Dog()
cat = Cat()

# Polymorphic function call
print(animal_sound(dog)) # Output: Woof!
print(animal_sound(cat)) # Output: Meow!
```

Run-time Polymorphism: Operator Overriding

```
class ComplexNumber:
```

```
    def __init__(self, real, imag):
```

```
        self.real = real
```

```
        self.imag = imag
```

```
    def __add__(self, other):
```

```
        if isinstance(other, ComplexNumber):
```

```
            real_part = self.real + other.real
```

```
            imag_part = self.imag + other.imag
```

```
            return ComplexNumber(real_part, imag_part)
```

```
        else:
```

```
            raise TypeError("Unsupported operand type")
```

```
# Creating objects
```

```
num1 = ComplexNumber(2, 3)
```

```
num2 = ComplexNumber(4, 5)
```

```
# Using the overloaded + operator
```

```
result = num1 + num2
```

```
print(result.real, result.imag) # Output:  
6 8
```

Run-time Polymorphism: Operator Overriding

- **Operator overriding** allows you to define how an operator behaves for user-defined objects.
- This enables you to use operators on objects of user-defined classes, and the behavior is determined at runtime based on the type of the objects involved.